Building the Credence Capability

Historical development notes

Andy Wilson | wilson.stats@gmail.com

Tao of RWD | tao-rwd.com

What this document is

A running record of how the Credence teaching app is being extended — written as a seminar rather than as changelog entries, because most of what we learned is worth teaching.

Three threads run through it:

1. **The Credence framework** — what it does, and the boundary between *realism* and *truth* that makes it work.
2. **The road to CausalMix** — where a hand-rolled conditional VAE runs out of road, argued from defects we actually hit rather than from the abstract.
3. **`survcausal-pilot`** — the larger project this feeds. Every entry closes by saying what it bought *there*, not just here.

Companion to: `tutorial/tutorial.pdf` (Tao of RWD).

Roadmap

Part I — The framework
Realism vs. truth; why the split matters.

Part II — Tier 0: making the claims true
Six defects, one of which was found *by* the fix for another.

Part III — Toward CausalMix
Post-hoc repair vs. architectural fix.

Part IV — `survcausal-pilot`
The larger project: what it is, what it still needs, and how this work supplies it.

Part V — Tier 1
$\tau(X)$ heterogeneity, an overlap dial, and factorial scenarios.

Part I — The framework

The problem Credence solves

You have an estimator. Does it work on *your* data? You cannot tell, because the individual causal effect is never observed. So the field validates on simulation — and simulation has its own problem:

- we hand-specify the data-generating process;
- a hand-specified DGP embeds parametric assumptions;
- those assumptions may quietly **favour some estimators over others**.

Credence (Parikh et al., ICML 2022) separates the two things a benchmark needs:

REALISM
learn $p(X \mid Z)$ from real data

TRUTH
choose τ, ρ, $f(X)$ yourself

The trick: because *you* chose τ, you can score any estimator. Because the covariates came from real data, the benchmark is not a toy.

The boundary that has to hold

The generator learns $p(X \mid Z)$ and **never sees the outcome**.

This is not a detail. If the generator knew the treatment effect, then "recovering" that effect downstream would prove nothing — you would be measuring how well information flows through your own pipeline, not whether an estimator works.

In the code, that boundary is a file boundary:

- `cvae_model.py` — realism. No Y, no τ, anywhere in it.
- `dgp.py` — truth. Chooses τ, ρ, $f(X)$.
- `realism.py` — evidence. Checks the first one actually worked.

Why it matters: Keeping realism and truth in separate files makes the boundary auditable. A reviewer can confirm the generator never touches Y by reading one import list.

The generative order looks backwards (and is fine)

In the world, X causes Z: sicker patients are treated differently. In the app we draw Z first, then $X \mid Z$.

That is not an error. $p(X, Z) = p(Z)\,p(X \mid Z)$ is a valid factorization of the same joint distribution — we are *sampling from the joint*, not asserting a causal direction. A conditional VAE finds $p(X \mid Z)$ far easier to learn than the reverse.

What it does cost us: the *strength* of observed confounding is inherited from whatever the real data happened to contain. It is not a dial.

Making it a dial — along with overlap — is exactly what Part V is about.

Part II — Tier 0: making the claims true

Where we started

The app told users its output was *"realistic and near-indistinguishable from the observed sample."*

It never checked. And underneath the claim were six defects.

What was wrong	Consequence
One gradient step per epoch	generator barely trained
Decoder round-trip never called	export did not match your schema
$f(X)$ selected columns by name	snake_case covariates silently dropped
$f(X)$ averaged on raw scale	dollars drowned out years
No seed on generation	nothing reproducible
No realism check at all	the headline claim was unfalsifiable

The through-line: an app that asserts its own quality without evidence is precisely the thing Module E trains participants to catch.

Defect 1 — fifty gradient steps

The training loop called `optimizer.step()` **once per epoch**, on the whole dataset at once. At the app's default of 50 epochs, that is **50 gradient steps in total**.

The defect: The "learned covariate structure" was closer to the untrained prior than to the data — while the interface promised near-indistinguishability.

Fixed: minibatching, with $\lceil n/\text{batch} \rceil$ steps per epoch, a seeded shuffle, and a per-observation ELBO so the reported number means the same thing at any batch size.

Measured, pneumonia cohort, 5 epochs: full-batch $= 1$ step/epoch, $-$ELBO 3.62; minibatch(128) $= 8$ steps/epoch, $-$ELBO $\mathbf{2.93}$.

Why it matters: We kept "Full dataset (the old bug)" as a menu option, so the workshop can *reproduce* the failure and watch the realism score degrade — rather than take our word for it.

The fix that mattered most — measuring realism

Three checks, weakest to strongest, all against **held-out rows the generator never trained on**:

Check	Question	Catches
SMD	are the means close?	location
KS distance	are the *distributions* close?	spread, skew, modes
Discriminator AUC	can a model tell them apart?	broken *correlations*

Why all three. A generator can match every marginal mean and get every shape wrong. It can match every marginal distribution and still destroy the *joint* structure — and only the discriminator, which sees whole rows, can notice that.

AUC ≈ 0.5 is the target. This is the rare model you want to perform badly. Participants find that disorienting, which makes it stick.

Why the holdout is not optional

These metrics must compare synthetic data against real rows the generator has **never seen**.

Scored against its own training rows, a flexible generator wins by **memorising**. That is:

- a meaningless result — it says nothing about generalisation; and
- a **privacy problem** — a generator that memorises patients can leak them, and participants upload their own files.

So Step 2 now splits before it trains, and Step 4 compares against the holdout.

Why it matters: Privacy metrics are one of the things CausalMix ships and we do not. The holdout is the minimum honest version of that idea.

Defect 10 — the diagnostic found a bug we had not listed

First run of the new realism check, after 150 epochs of proper training:

Discriminator AUC = 1.000

Perfect separation. The obvious reading — "the generator is hopeless" — was wrong.

Covariate	Real	Synthetic
`priorPneumonia`	$\{0, 1\}$	$[-0.30,\ 1.35]$ continuous
`priorVaccine`	$\{0, 1\}$	$[-0.25,\ 1.35]$ continuous

The decoder round-trip only ever handled `object`/`Categorical` columns. **Binary and integer covariates were exported as continuous relaxations.** The classifier was not detecting anything subtle — it was asking "is this value exactly 0 or 1?"

Why it matters: The diagnostic paid for itself on its first run — by finding a defect that was not on the list it was built to check.

Sampling, not rounding

Fixing it raises a genuine statistical choice. Given decoder output implying $P = (0.62, 0.31, 0.07)$ across three categories:

argmax / threshold
Always take the most likely.

- every row $\rightarrow$ category 1
- rare categories **vanish**
- synthetic data less variable than real

sample
Draw with those probabilities.

- frequencies come out right
- marginal distribution preserved
- costs nothing

What we need to preserve is the **marginal distribution**, not the most likely value of any single row. So we sample — categorically for categories, Bernoulli for binaries, round-and-clip for integers.

Measured: with a true rare-level share of 15%, sampling returned **16.7%**; argmax returned **0%**.

Defect 4 and 5 — what $f(X)$ was

$f(X)$ is the prognostic index: how covariates move $Y(0)$. It *is* the observed-confounding mechanism. It was one line:

```
numeric = [i for i, name in enumerate(names) if '_' not in name]
hX = X[:, numeric].mean(axis=1)
```

Two independent defects.

1. **Selection by name** — keeps columns whose name has no underscore. Meant to skip one-hot dummies; also drops any covariate named `prior_vaccine`. Still *exported*, so it looked like a confounder and influenced nothing.
2. **Raw-scale averaging** — an unweighted mean across units is decided by whichever column has the biggest numbers.

Why it matters: Both built-in datasets use names without underscores — so it survived. The first participant to upload their own file would have hit it.

. . . and what $f(X)$ is now

Every encoded column participates — selection by *name* is gone. Each is standardised on the **training** statistics, weighted, and rescaled to a stated magnitude:

$$f(X) \;=\; c \cdot \sigma_Y \cdot \frac{\sum_j w_j \, z_j}{s}, \qquad z_j = \frac{x_j - \bar{x}_j}{\mathrm{sd}_j}$$

- **unit-invariant** — rescale earnings and $f(X)$ does not move;
- **weights visible in the UI** — weight 0 means cannot confound;
- **magnitude interpretable** — c is in outcome standard deviations.

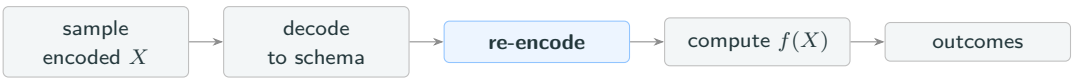
Why it matters: Direction is a stated choice (w); magnitude is a stated parameter (c). Turning c into a dial is Tier 1.

The property that makes a benchmark usable

The exported covariates must fully determine the potential outcomes (up to U and noise).

If the outcome secretly depends on something we did not export, then *no* analyst can adjust for it, *every* estimator looks biased — and the workshop teaches a false lesson about the methods instead of a true one about the data.

This forced a step the old pipeline did not have:



Without the re-encode, a categorical's *exported* value only loosely tracks the value that actually drove the outcome.

Two seeds, on purpose

Generation used unseeded draws: every click produced different data and nothing was reproducible.

The fix is not one seed but **two** — training and generation, kept apart:

- redraw a cohort *without* retraining the model;
- retrain *without* moving the cohort;
- so you always know which change caused what you are looking at.

This mirrors `survcausal-pilot`'s own convention, which seeds the cohort draw and the estimator fits from **disjoint streams** for exactly this reason (`sseed` / `eseed` in `run_pilot.R`).

Why it matters: Attribution. The same principle as the pilot's factorial design: move one thing at a time, so any difference has a single cause.

The gate: what we refuse to ship without

Ten checks, runnable with `python3 tests/test_tier0.py` — no pytest, so a participant can run it on a workshop laptop. The one that matters most:

The truth gate. With $\rho = 0$ the only path from Z to Y other than the effect runs through the exported covariates. A correctly specified adjustment *must* recover τ.

	estimate	bias
true τ	-5.000	—
naive difference	-4.349	$+0.651$
g-computation	$\mathbf{-4.977}$	$\mathbf{+0.023}$

(tolerance ± 0.130, four Monte Carlo standard errors)

Its complement. At $\rho = 0.8$: adjusting for X leaves bias $+0.284$; adding U brings it to -0.015.

Why it matters: If the truth gate fails, the generator is wrong and nothing downstream can be trusted.

Tier 0 — result

Discriminator AUC	before	after
undertrained generator (1 full-batch epoch)	0.852	
properly trained generator	1.000	$\mathbf{0.602}$
app flow, pneumonia cohort	—	$\mathbf{0.579}$

The $1.000 \rightarrow 0.602$ column is the type-aware decoder (defect 10). The 0.852 figure is what an undertrained generator still scores *after* that fix — which is why both numbers are reported: the **improvement** is the teaching artifact, not the absolute level.

Verdict at 0.579: *"Indistinguishable — the discriminator is near chance."* The claim the app was already making is now one the app can defend.

Part III — Toward CausalMix

What CausalMix is

CausalMix — Zhang, **Parikh**, Naimi, Nabi, Kim, Lash (arXiv:2603.03587, March 2026).

A conditional VAE with:

- a **mixture-of-Gaussians latent prior** — real covariates are multimodal and clustered; a single Gaussian smooths that away;
- **data-type-specific decoders** for continuous, binary and categorical columns;
- an **overlap regularizer** shaping the propensity-score distribution;
- overlap, confounding strength and effect heterogeneity varied **independently at design time**.

The decisive fact: Harsh Parikh is first author of Credence *and* an author of CausalMix. This is not a competing school — it is Credence's own lineage, four years on. Adopting it is a continuation, not a retraction.

Defect 10, revisited — the case for typed decoders

Our fix for binary and integer columns is **post-hoc repair**: the network is trained as if every column were continuous, and we discretize its output afterwards.

What we did

- MSE loss on every column
- Bernoulli-sample binaries after decoding
- round-and-clip integers after decoding

Works. AUC $1.000 \rightarrow 0.602$.

What it cannot fix

- the *training signal* still treats a $0/1$ flag as a continuous target
- so the *joint* structure between a binary and a continuous covariate is learned only approximately

The architectural fix is to give the model a per-column likelihood and decode each type natively — which is exactly what CausalMix does.

Why it matters: We can now argue for Tier 2 with a *measured* number rather than an appeal to authority. That is a better reason to adopt something.

Caveats we are not going to forget

1. **Maturity.** March 2026 preprint, no visible CI, not on PyPI, effectively one maintainer. Fine for teaching; pin a commit and validate before any number reaches a paper.
2. **Deployment weight.** shinyapps.io already strains with `torch`. Mitigation — correct regardless of backend — **ship pre-fit generators; let the app only sample**.
3. **No survival outcomes.** Neither Credence nor CausalMix does time-to-event. For `survcausal-pilot` that is an integration, not a swap.
4. **The parametric assumption moves; it does not vanish.** Realism is bought for $p(X, Z)$ only. $f(X)$ and τ are still ours.
5. **Self-flattery risk.** A generator fit on the same data used for the benchmark can favour estimators whose inductive bias matches the generator — a VAE's smooth latent structure flatters smooth-function learners.

Say caveat 5 out loud in the workshop. Credence's entire pitch is that hand-specified DGPs may inadvertently favour certain estimators. The same critique applies to us.

Part IV — `survcausal-pilot`

The larger project

`survcausal-pilot` — a pilot for a causal-estimator extension to Diana Shamsutdinova's `survcompare` package. *Shamsutdinova–Shuryak–Yang collaboration.*

Two questions, deliberately not conflated:

What will happen?
Prognosis. Ranks patients by risk. Scored on discrimination and calibration.

What should we do?
Benefit. Targets the treatment effect. Scored on RMSE against a known $\mathrm{ATE}(t)$.

A patient can be high-risk and gain little; lower-risk and gain a lot. A purely predictive model conflates the two.

The causal question needs known truth — which only simulation can give. That is where this work connects.

The pilot's headline result

A 2×2 factorial Monte Carlo (50 replicates/cell), crossing confounding shape with hazard shape *at a single fixed effect size*, so any difference has one cause.

relative RMSE	Cox-PH	RSF	CSF	CAST (spline)
Linear / PH	**0.087**	0.365	0.230	0.224
Linear / non-PH	0.897	0.611	0.358	**0.352**
Nonlinear / PH	0.466	0.452	0.186	0.180
Nonlinear / non-PH	1.148	0.764	0.250	**0.239**

Under simple confounding with proportional hazards a correctly specified Cox model wins outright and causal forests add nothing. Break *either* assumption and the parametric methods collapse while CSF and CAST hold.

The lesson: the right method is data-dependent. A benchmark reporting one regime would mislead.

What the pilot says it still needs

From the pilot's own vignette and README — these are not our inventions:

- *"a data-generating process with genuine **covariate-driven effect modification**"* — named as a natural extension;
- *"A future data-generating process can add genuine covariate-level heterogeneity; the machinery here would estimate it unchanged"*;
- *"**Causal scoring needs known truth.** RMSE against the true $\mathrm{ATE}(t)$ is only possible in simulation, which is why the package ships simulation DGPs with a treatment arm and oracle effect."*

So the pilot's causal-validation story rests entirely on **simulated DGPs with oracle truth** — and its one soft spot is that those DGPs are hand-specified, in exactly the manner Credence was written to criticise.

A reviewer will ask.

Connection 1 — $\tau(X)$ is the pilot's named gap

The pilot's risk/benefit quadrants (Figure 3) make their point via a *non-monotone* relation between risk and benefit, because the treatment hazard ratio depends only on time — there is **no covariate-driven effect modification** in the cohort. The vignette concedes this twice.

Tier 1's $\tau(X)$ knob is a prototype of exactly that missing DGP feature.

	app (Python)	**pilot (R)**
cost of one run	seconds	1–1.5 hours
feedback loop	immediate	overnight
stakes	teaching	publishable numbers

Why it matters: Prototype $\tau(X)$ where a run costs seconds; port it where a run costs ninety minutes. The app is the cheap end of the same question.

Connection 2 — the overlap dial feeds a dormant diagnostic

The pilot *already computes* positivity diagnostics: `pct_extreme` and `pct_clipped` in `run_causal()`, carried through into the summary table.

But nothing in the study ever **varies** overlap. So across all four design cells those columns read:

`0.00000, 0.00000, 0.00036, 0.00047`

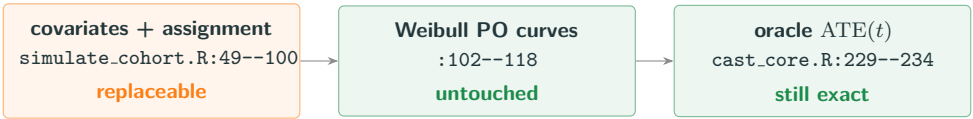
A diagnostic that never moves is a diagnostic that teaches nothing.

Tier 1's overlap slider turns a passive column into a **third design lever** — and it is the axis most likely to separate doubly-robust CSF from the plug-in learners in a way non-proportionality alone does not.

Why it matters: Positivity is a first-class identification assumption in Modules C and D. No current teaching asset lets anyone *move* it.

Connection 3 — the oracle survives semi-synthetic covariates

The strategic prize: anchor the pilot's covariates on a *real* cohort while keeping the known $\mathrm{ATE}(t)$. The seam already exists.



Why the oracle survives. `true_survprob_ate()` computes $\tau(t)$ by averaging $S_1 - S_0$ *over the covariate sample*. It never assumes the covariates came from a known parametric law — only that the potential-outcome curves are known for each row.

So a semi-synthetic covariate sample still yields an exactly computable oracle. That is what makes this viable rather than merely appealing.

The honest limit

If we anchor covariates on a real cohort, the survival mechanism — the linear predictor `lp_surv` and the treatment model — is **still a hand-specified function** of those covariates.

Realism is bought for $p(X, W)$. It is not bought for the outcome model.

This must be stated in the pilot's README if that work ships. Otherwise the pilot would over-claim in precisely the way this dev log spent Part II faulting the app for.

And one more, for the same reason: the pilot's data note currently reads *"All cohorts are simulated from a known generative model. No patient data are used or distributed."* That sentence becomes false the moment a real cohort is touched — and must be rewritten *before*, not after.

Part V — Tier 1

Tier 1 — what we are building

Tier 0 made the app's existing claims true. Tier 1 adds the capability that was genuinely missing — and it is the part that maps onto `survcausal-pilot`.

1. $\tau(X)$ — **heterogeneous effects.** Constant τ is why Step 4 cannot currently tell TMLE, AIPW and g-computation apart: they all recover the same number. With $\tau(X)$ the comparison acquires teeth.
2. **Overlap as its own dial**, separate from ρ. Hold τ and confounding fixed, degrade overlap, watch estimators fail.
3. **Factorial scenario grid.** Cross $\{\text{overlap}\} \times \{\text{confounding}\} \times \{\text{heterogeneity}\}$ and summarise in one table — mirroring `run_pilot.R`'s design so that one cause moves at a time.

All three now built; twelve tests green.

$\tau(X)$ — and the property we designed around

$$\tau(X) \;=\; \tau_0 \;+\; \kappa \cdot \sigma_Y \cdot m(X)$$

$m(X)$ is a *centred* covariate index, so $\mathbb{E}[\tau(X)] = \tau_0$.

Turning heterogeneity on does not move the average effect. The number the user typed stays the ATE; only the spread changes.

That is deliberate. Without it, a participant raising κ would watch the ATE drift and could not tell whether the estimator had broken or the target had simply moved — the same attribution principle as the two seeds, and as the factorial.

Measured: $\kappa = 0$ gives ATE -5.000, $\mathrm{sd}(\tau) = 0.000$; $\kappa = 1$ gives ATE -5.000, $\mathrm{sd}(\tau) = 0.852$.

Why it matters: This is the pilot's named v2 gap. `survcausal-pilot`'s vignette concedes twice that its cohort has no covariate-driven effect modification; prototyping it here costs seconds per run instead of ninety minutes.

Why constant τ was holding the app back

With a constant effect, g-computation, IPW, AIPW and TMLE all recover the *same number*. A participant comparing them learns only that they agree — the problem is not hard enough to separate them.

It also makes the clinically important question unaskable: *does this work?* versus *does this work* ***for this patient****?* Only the second requires τ to vary.

With $\tau(X)$ live there is finally something for a CATE estimator to find. An interacted outcome model recovers the per-patient effect at $r = \mathbf{1.000}$ on our own DGP — which is the right answer, since that model is correctly specified here. The interesting version of this question starts when it is not.

Why it matters: This is the same quantity the pilot's risk/benefit quadrants are built from. The pilot can already *estimate* a CATE; what it lacks is a DGP where the true CATE varies.

The overlap dial — and the trade it forces

Learned (Tier 0)
draw Z, then $X \mid Z$

- confounding inherited from real data
- realistic
- not adjustable

Designed (Tier 1)
draw X, then $Z \mid X$

- logit $e = \alpha + \lambda m(X) + \rho U$
- λ is a dial
- arm structure is ours, not the data's

You cannot have both from a conditional VAE. $p(X \mid Z)$ and a controllable $e(X)$ pull in opposite directions.

Getting realistic covariates *and* a designed propensity out of one model is exactly what CausalMix's overlap regulariser is for — the second concrete, measured argument for Tier 2.

What λ actually does

λ	propensity range	% extreme	reading
0	$[0.565,\ 0.565]$	0.0%	randomised trial on realistic covariates
1	$[0.136,\ 0.895]$	0.0%	real confounding, comfortable overlap
3	$[0.003,\ 0.999]$	33.6%	confounding *and* positivity failure

Note the middle row. **Confounding and positivity are separate failures.** At $\lambda = 1$ the naive estimate is badly biased ($+0.68$) while every propensity stays comfortably interior — exactly the setting where adjustment is supposed to work, and does.

A dial that conflated the two would teach the wrong lesson. This one does not, and the test suite asserts that it does not.

Why it matters: Positivity is a first-class identification assumption in Modules C and D. Until now no teaching asset let anyone *move* it.

The factorial grid

Three axes crossed, everything else held fixed — generator, covariates, τ_0, n. Twelve cells. Selected rows:

overlap	effect	hidden	naive bias	g-comp bias	% extreme
randomised	constant	none	-0.013	-0.030	0.0%
randomised	varies	none	-0.086	-0.047	0.0%
mild	constant	none	$+0.684$	-0.042	0.0%
mild	varies	present	$+0.994$	$+0.072$	0.2%
poor	constant	none	$+1.166$	$+0.037$	33.6%
poor	varies	present	$+1.900$	$+0.080$	35.3%

Each axis moves something the others do not — positivity tracks λ alone, effect spread tracks κ alone, residual-bias-after-adjustment tracks ρ alone. **That separation is asserted by the test suite**, because a factorial whose axes move together is not a factorial.

Why it matters: Directly mirrors `run_pilot.R`'s 2×2 at fixed effect size. Same discipline, three axes instead of two, seconds instead of hours.

A limitation the grid exposed

Building the dial surfaced something about the generator.

The prognostic index has **unit** standard deviation on real training data by construction. On synthetic covariates it comes out at $\mathbf{0.852}$.

The CVAE, trained with a squared-error reconstruction loss, systematically **shrinks variance** — visible too in the raw ranges: real age spans $[7,\ 116]$, synthetic $[36,\ 98]$.

Practical consequence: the overlap dial runs about **15% weak** — $\lambda = 3$ behaves like roughly $\lambda = 2.6$. This is now a tripwire test rather than a footnote: if the ratio collapses further, the grid's axis labels stop meaning what they say.

Why it matters: Third measured argument for Tier 2: a mixture-of-Gaussians prior exists precisely to stop a single-Gaussian latent from collapsing the spread of the data it is modelling.

Where Tier 1 leaves us

The app can now

- vary the effect across patients
- dial positivity independently of confounding
- run a 12-cell attributable factorial
- score two estimators against known truth in every cell

Three measured arguments for Tier 2

1. typed decoders — defect 10
2. overlap regulariser — the trade on the previous slide
3. mixture prior — variance shrinkage

None of these three was an appeal to authority. Each came out of a number we watched go wrong.

Next: Tier 2 puts the generator behind an interface and ships pre-fit artifacts; Tier 3 is the `survcausal-pilot` integration — real covariates, oracle $\mathrm{ATE}(t)$ intact.

References

- Parikh H, Vajao C, Xu L, Tchetgen Tchetgen E (2022). *Validating Causal Inference Methods*. ICML (PMLR 162). https://arxiv.org/abs/2202.04208
- Zhang Q, Parikh H, Naimi A, Nabi R, Kim C, Lash T (2026). *CausalMix: Controllable Generative Sandbox for Causal Inference*. https://arxiv.org/abs/2603.03587
- Yang E, et al., Shuryak I (2025). *CAST: Time-Varying Treatment Effects*. https://arxiv.org/abs/2505.06367
- Shamsutdinova D, Shuryak I, Yang E. `survcausal-pilot`. https://github.com/ishuryak/survcausal-pilot
- Kingma DP, Welling M (2014). *Auto-Encoding Variational Bayes*. ICLR.
- Dang LE, Balzer LB (2023). *Start with the Target Trial Protocol, Then Follow the Roadmap for Causal Inference*. Epidemiology 34(5):619–623.